Software Engineering Assignment

Rasmus Kovács

July 2026

1 Introduction

My research area is in robotics and automatic control. So I do not do any AI/ML research but of course use the tools. In particular, my project so far is about designing adaptive control algorithms for a robotic planar gripper. The idea is that an adaptive controller would be able to adapt itself to different material online while grasping and hence give fast and accurate control across different objects without requiring retuning.

In terms of software, I need to implement my control laws in a sensible way to try it out on actual hardware. The hardware consists of sensors for reading the motor encoder and for force feedback of the grasping force of the gripper. These are read asynchronously at different frequencies. The gripper I have access to is custom built and is unique in the sense that it has a very accurate and high frequency force sensor together with direct voltage control to the motor. With this hardware available I have an interesting opportunity to investigate what practical modifications needs to be added to a basic Model Adaptive Refernce Control scheme in order to perform well on hardware with sensor noise, unmodeled dynamics and discretized states with limited bandwidth.

2 Lecture Principles

2.1 Technical Debt

I thought I would share a little story on my own experience of technical dept. As a PhD student without a background in software engineering I often use generative AI for coding. With tools like chatgpt and codex, I can quickly prototype and explore the performance of control algorithms and get a feeling for how they work.

For example, as I write in the introduction, I am controlling a planar gripper. This gripper was given to me and developed by another PhD student of my supervisor. He coded it all in c++ as well as python within the robot operating system ROS. With little experience in ROS and c++ I often relied on LLMs to implement the math I derived. Then I would look it through and observe that the math looked correct and then later try it out, firstly in a simulated environment, and then on hardware. This means that I never really took the time to understand all of the code that was in the repo when I got. I'm not convinced it would be a good use of my time as a control engineer to spend time on this but as the project gre larger things got a bit out of control. I did not really plan beforehand of what parts I wanted to include in the future, how different configurations of control laws would be incorporated but instead ended up with multiple copies of the same file, with slight changes in each file. Then sometimes I would make some structural changes in some files, and forget to change it in another and it was simply all a mess. It definitely hindered me from being productive, since I had no sense of what I already tried and I had a very hard time moving forward in a structured manner. I think this clearly illustrates a drawback of me using to much LLM in this setting, and maybe if I took the time to think this through before starting to implement everything it would have saved me quite some

time later. I also would like to add that at the time I only used ChatGPT in the browser, so it had no real sense of the full context.

Luckily, a college of mine, who has a background in software engineering, is working on a python based repo designed for controlling robots in a nice and intuitive fashion. So we decided to add control of grippers (and not only full body robots) in this library. A lot of the backbone of the code base was already there, and at this point I had discovered that we have access to codex through the university. So with the help of codex I tried to understand how all of the structure of his library were built so that I could implement things in a nice and structured way. I think that this approach worked much better and now I am more confident of the code that I have. However, I would still say that there is a lot of code that I have never looked at that makes the thing running, and I am concerned about this. However, it would take me forever to understand it all, and I for sure would not have been able to try out so much of different interesting techniques and control laws if I did not use AI. So I believe there is a tradeoff in understanding of the code and how fast things can move. And as a control engineer, I don't want to spend to much time coding and thinking about this. However of course I need to be confident that the code actually does what I think it does.

2.2 (Junior) roles contract, thin out, disappear

As I looked through the material again when writing this, one thing stood out to me that I have seen at other places before as well. In "Agentic Software Engineering Will Eat the World: AI-based Systems as the new OS of Society" there is a bullet point that states "(Junior) roles contract, thin out, disappear". And I don't really understand how this is supposed to work long term.

I understand that a lot of the easier tasks, that someone in their early career would do as a software engineer, can be done easily with the help of a coding agent, and maybe even with better results if prompted by a more senior developer that understands the code base in and out. But in 20 years when this senior retires, there may be a struggle to maintain the project since the knowledge was not passed on. To me it is obvious that there is a point to hire junior programmers, not necessarily because they can do the work better, but otherwise there will be no one in the future that is capable of solving problems and developing advanced code. Perhaps if AI is good enough someone will be able to keep prompting it to fix what is needed, but then it seems like we lost the human understanding, and to me that is very important.

I guess if fewer junior roles are available, there must at least be some other way of passing the knowledge to the next generation. Maybe through more code review, testing or something else. But some kind of learning and experience is of course required to become the senior developer that is still demanded according to the plot in the slide.

3 Guest-lecture Principles

3.1 Work load balance

Another thing that I found really interesting in the course was the work balance between easy, medium and hard tasks.

The idea mentioned is that one should balance the work load with 75% easy tasks, 20% medium tasks and 5% hard tasks. My interpretation is that this gives the brain some sort of optimal workload. The takeaway for me was that even though the easier tasks can feel like a waste of time, like filling in some spreadsheet or doing some tedious calculations, it gives the brain time to recover in a sense and be able to focus energy into the harder tasks when needed. Then the medium tasks are supposed to be a bit more challenging but in a way so that they feel like it is possible to accomplish them with the knowledge you have, but it might require

some effort. Then the last 5% should be really challenging, maybe requires learning new things, discussing with colleges and are more long term goals or things we try to achieve.

By keeping the balance like this, we keep feeling good because for the most part we manage to accomplish things and keep making progress. Personally I feel like I have outsourced a bit to much of the easier tasks for AI. I keep thinking that it is stuff I already know and I think about how much time I could save if i let codex/chatGPT implement it for me, and usually the code (at least short term) becomes better. But what I failed to think about before is how this obscures the workload so that I spend more of my time in the harder tasks, and I genuinely think that it effects my well being in a negative way. The constant struggle of feeling like I am making progress or contributing to something useful, and not just prompting, is making me tired. So since that lecture I have been trying to force myself to do the tedious tasks myself and not always really on AI to do it for me. Now the stress instead comes in that I don't feel as productive when I don't use it, but I keep reminding myself of this lecture and believe that it is for the best.

I think that there are a lot of challenges with AI that is not talked about so much, and that this is definitely one of them.

3.2 Requirements and specifications for AI-assisted development

One principle from the requirements-engineering guest lecture that I found especially relevant was that one should understand the problem before building the solution. The lecture also distinguished between the problem space, which concerns what a system should do and why, and the solution space, which concerns how the system should do it.

Since I got my hands on codex through the university I have experimented quite a bit with it for personal projects, as well as in my research, to learn about how one can use it and how it performs. From what I have seen, I think that it is clear that it is much more suitable for the lower level tasks of the V, than it is for designing a whole progress.

The most successful I have been, is when I created a full description of the project, specifications of the program, a progress-line for what should be the first focus etc etc and then gave this as a readme available in the context of codex. So, Basically I myself take care of collecting requirements, system specification as well as system testing on the other side of the V. Then I let the AI do design in terms of choosing what packages to use, how the code is structured, do the actual coding and so on. And it has actually been quite successful in creating useful programs. I managed to create a web-app where I now overview my economy, including stocks, monthly budgets as well as budgets for projects/trips, and it summarizes how much I spend on different categories each month. And it was all quite effortless and something I would never pull of on my own.

However for another project. I more in general described what I wanted and let it do its thing. What happened was that it started diverging from the original idea a bit, and it was very hard to fix things post-hoc because some implementation choices was just straight up bad for the app I wanted to create. And since this was not really specified concretely before hand it could not create code that fulfilled this purpose.

So I think my lesson learned is that the AI is only a tool that can do what you ask it. And it will only be as good as the specifications you give it. At least if you have something specific in mind, I think it is clear that it is good to have this written down concretely before you let the AI loose.

With this said, the specifications can of course also be created with a LLM. And actually I often used Chat-GPT to write the specifications down in very nice and structured way based on the prompt and ideas I fed it. Then I read through it and made sure that I agreed with the plan before giving this to codex.

Now this was only for personal projects and nothing was every safety critical so I did not have to think about this so much. But I definitely think there is potential in developing stuff very

quickly with these tools, but one has to be careful what part of the development is outsourced.

4 Data scientists, software engineers, and AI engineers

I think the data scientist would focus more on the models, tuning hyperparameter for performance and evaluating them on different datasets. While the software engineers more likely would simply implement it and use it for something more practical than for the sake of benchmarking, with a stronger focus on the end product, respecting customers demands, making sure it is safe etc etc. To be honest, I don't really have a view on this from before so I am not really sure. I think the distinction is sensible as a description of different specializations, although it should not be treated as a strict boundary. Some people work across both areas, and the chapters themselves present the distinction as a generalization.

I completely agree that ML requires a system-wide view. When doing an exercise at school we often focus on simply getting a required score on some benchmark but nor really focusing on user experience using the algorithm, or fail safety etc. I don't do ML, but the adaptive controllers I develop learn from data, and it is very dependent on what sensors I use, what material I try to grasp with the gripper, the implementation for achieving periodic control loops and not only the equations that I can prove stabilizes the system. Here the scientist in me that develop math must work with the engineer that implements things and get things actually running. If I only tweaked my parameters to work for some specific and nice behaving material it may struggle to generalize. With a more system-wide view I can get a feeling of how to tune my parameters to achieve both good performance but also robustness. I think a system-wide knowledge is necessary to tweek parameters to achieve both criteria in a nice way. I think this might translate into the same conclusion regarding ML.

I think that it is always good to have a broader understanding than your own narrow field, and I think that this actually will strengthen the performance in the field in question also. That does not mean I should know everything about everything as that would take forever, but for them to at least be able to understand the others view on things I think is very valuable.

When it comes to AI engineering, I think about it, based on the text as the practice of using AI to build systems ti has AI as an important role in it. It include things like prompt design, integrating pertained models and creating tools etc. I think that with things like foundation models and AI coding assistants it is easier to cross boundaries and for both of the parts to do work that the other part would traditionally do. This I can also relate to in a sense that I am a control engineer, but still easily was able to port a driver in cpp into python for a Ethernet communication force/torque sensor, that I would have had to outsource probably otherwise to do this in a reasonable time frame.

5 Paper Analysis

5.1 Paper 1: Developer Experiences with a Contextualized AI Coding Assistant: Usability, Expectations, and Outcomes

I read a paper called *Developer Experiences with a Contextualized AI Coding Assistant: Usability, Expectations, and Outcomes* [1]. As the title suggests it was a study on letting users try out to work with a contextualized AI coding assistant called StackSpot AI. The idea is that there is, say at a company, some sort of database or context that is fed to an AI that gives internal information to the AI during each prompt. Say that the company has some internal python libraries, or a lot of documentation on how things work, guidelines or something else that is not general knowledge; then this would be known by the AI thanks to the context and hopefully it can give better results than a general LLM would. It seems like the users in general seam to appreciate this kind of context based LLM but there are a few caveats. It seems like it

was working bad if multiple contexts were introduced. Also not all sources could be used, for example one could not have GitHub repo as context, which I believe would make very much sense.

The first thing that I started to think about when reading this paper was that this seems pretty widely done already, and then realized that this paper is two years old and it is really crazy to me how much happened in this short time. I think a context based AI is something that is used a lot. However I'm not sure I have come across a non-personalized context setting to work with but I've heard about this at some companies, like IKEA doing something that sounded like this. However as I already mentioned, I use context based AI almost daily when I code and I find it very helpful.

I think the idea of this context based thingy could be very useful, say that me as a TA create a context for a course that we teach, maybe I add some books as context, some of the exercises with solutions, the lecture slides, and then I would also add a readme file that specifies how the AI should respond when a student asks questions. Maybe I want the AI to certain hints in some of the questions when the student asks for help, or maybe I could just specify that the AI should never give the complete answer unless some certain criteria and then maybe we could have the time table for when we have exercises directly in the context so that it could tell student to go there.

I find this idea very interesting because I guess most students use AI to solve exercises, and I believe most use some online tool like ChatGPT. And when you are tired or just want to get done, it is so easy to just give in and let it solve it for you, but if you used "our" AI interface instead which of course would use, say ChatGPT in the background, maybe the response could become more like a TA would do it and not instantly give the answer to student. And I think that as a student, I would really appreciate this. I bet most people want to learn, but it is hard when there exist a good shortcut. Maybe people would consider using this instead.

I think, regarding my research, I would be the user of this kind of context as I am not doing any development in this field myself. But I think it would be genuinely useful for my research to keep the sources that I actually use in a context so that when i discuss with it, it can actually have the original papers or code bases or whatever as context.

I think it would actually make sense, for example when writing a paper, to introduce some different papers you read as context so that you and the model is on the same page when it comes to what knowledge is new and relevant. So maybe I could try thinking about this more than just uploading files every time I want to ask something more specific.

5.2 Paper 2: From Requirements to Code: Understanding Developer Practices in LLM-Assisted Software Engineering

The second paper I read is called *From Requirements to Code: Understanding Developer Practices in LLM-Assisted Software Engineering* [2]. This paper focuses on how well written requirements for some code/software can be translated into code. Is it enough to simply give the requirements to an LLM and expect it to deliver a working product, or does one have to do more than that?

One of the main findings seems to be that it is not enough to give requirements to an LLM and let it decide. Usually the code it writes is not really useful since going from requirement into code often requires some implementation decisions. This sort of decisions may traditionally be done without to much thinking from the programmer, based on the rest of the code structure, company policies/guidelines for writing code, packages that are familiar to the coding team etc and is not something that the AI is aware of.

Based on the paper, it seems like a lot of people get a lot more useful results if they first breakdown the requirements into more concrete coding tasks, mostly manually, and then give these to the LLM to work from. In this way larger part of the design is done by a human. It also

seems like the smaller tasks you divide it into, the more useful the LLM tool is at completing it in a immediately useful state of the code.

The paper also talks about reusing older conversations when asking a question that is related. They reason that this particular conversation already has the context necessary to answer the question and therefore a good output can be generated faster.

I think that I can relate to a lot of things written in the paper in my own research. I feel like when I ask for something to broad, Codex for example can make a lot of assumptions and introduce a lot of new libraries and introduce feature that was not really my intent. Sometimes they are useful, but when more and more stuff enters the code base, it starts feeling messy and incoherent and it is hard to navigate the code among all the fuzz. By instead narrowing down what is actually needed and more precise steps on what I want to achieve, I feel like I generally have a higher success rate when employing these tools.

Regarding integration into an AI-intensive project, I tried to recreate a game called time guesser but for Skåne only, using only codex. We started by just writing down the requirements, e.g. how the game should work and what we wanted to do. This was a project with my little brother with zero coding experience and hence I though it would be nice to see what happened if we only wrote and did things that made sense to him. The early results were actually really bad and we had to spend a lot of time afterwards, really pointing at what went wrong in order to fix things. I think that this is a perfect example of how it can go wrong when only specifying the high level requirements. I can't really think of a way on how my research could be applied here.

When it comes to how i could adapt my research, I think my main takeaway here is that I am more convinced now that simply specifying what I want to achieve is not enough to just give this to a LLM, but instead I should really think stuff through beforehand (as a decent research should) and then maybe I could let an LLM accelerate the workflow.

6 Evidence, originality, and GenAI-use transparency

The only sources I relied on are the lecture slides, the book for question 4 and the two papers for question 5.

I used ChatGPT to validate my answers that I wrote here, it often had some comments in how I formulated things, what part of the text were week and what parts were good. Based on this feedback I reformulated or added insights in the text. I also used ChatGPT to choose papers for me for question 5. In that prompt I gave it the exercise instruction and to as well take my research into account so that I could write something meaningful. However this is no entirely easy as I do not do research in AI.

References

[1] Gustavo Pinto et al. "Developer Experiences with a Contextualized AI Coding Assistant: Usability, Expectations, and Outcomes". In: *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI*. CAIN 2024: IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI. Lisbon Portugal: ACM, Apr. 14, 2024, pp. 81–91. ISBN: 979-8-4007-0591-5. DOI: 10.1145/3644815.3644949. URL: https://dl.acm.org/doi/10.1145/3644815.3644949 (visited on 08/07/2026).

[2] Jonathan Ullrich, Matthias Koch, and Andreas Vogelsang. *From Requirements to Code: Understanding Developer Practices in LLM-Assisted Software Engineering*. July 10, 2025. DOI: 10.48550/arXiv.2507.07548. arXiv: 2507.07548[cs.SE]. URL: http://arxiv.org/abs/2507.07548 (visited on 08/07/2026).